

Part 1

HTTP: where everything starts

What it is, why it exists, when to use it, when not to, the trade-offs, and how to build it in Go. Every code sample in this PDF compiles and is covered by tests.

Covers	Client-server model · statelessness · TCP/TLS/HTTP versions · message anatomy · headers · methods and idempotency · CORS and pre-flight · status codes · caching · content negotiation · compression · keep-alive · multipart and streaming · HTTPS
Sources	Sriniously video 5 ("Understanding HTTP for backend engineers") transcript · GitHub repo DsThakurRawat/Backend-from-first-Principle (26-topic syllabus) · your own auctionEngine Go repo · gaps filled from RFC 9110/9111/9112 and the Fetch spec
Runnable code	backend-course/code/ch05_http/ (raw TCP server, full net/http server, HTTP client; 13 tests, race-detector clean)
How to read	Each topic has a study card (what / why / when / when not / pros / cons / how), diagrams, Go code, a pointer to the matching file in auctionEngine, and interview angles. Part 1 ends with 30 interview questions including scenario questions.

REMEMBER

Honest scope note. The transcript I had was for video 5 only, and the Vercel site and YouTube were blocked in this environment. The rest of the syllabus comes from the GitHub repo's folder names, so later parts will be checked against those folders (and your transcripts when you paste them). Points where I correct or refine something the video said are marked **CORRECTION** / **NUANCE**.

00 · HOW THE SERIES IS ORGANISED

The complete roadmap

The GitHub repo lists 26 topics. Below is each one mapped to a PDF in this series, plus a second list of topics that interviews ask about but the course does not cover in depth. Nothing here is skipped: if a topic is not in the course, it is in the gap list.

#	Course topic (from the repo)	Part	Why it matters
1	HTTP and CORS · this PDF	1	Every other topic rides on HTTP
2	Routing in the backend	2	Mapping method + path to code
3	Serialization and deserialization	2	JSON, validation of untrusted input, protobuf
4	Authentication and authorization	3	Sessions, JWT, OAuth2, RBAC, password hashing
5	Validations and transformations	2	Never trust the client
6	Controllers, services, repositories, middlewares, request context	2	Layering: where code lives
7	API design (REST)	4	Resources, pagination, versioning, idempotency
8	Databases with the backend	5	SQL, indexes, transactions, isolation, pooling, migrations
9	Caching	6	Redis, cache-aside, stampede, invalidation
10	Task queues and background jobs	7	Retries, outbox, SKIP LOCKED workers
11	Full-text search (Elasticsearch)	6	Inverted index, search as a read model
12	Error handling and fault-tolerant systems	8	Timeouts, retries, circuit breakers, bulkheads
13	gRPC and inter-service communication	9	Protobuf, streaming, REST vs gRPC
14	Production configuration management	8	12-factor, secrets, env vs files
15	Logging, monitoring, observability	8	Logs, metrics, traces, SLOs
16	Graceful shutdown	8	Draining, SIGTERM, in-flight work
17	Backend security	3	OWASP top 10, CSRF, XSS, SQLi, SSRF, headers
18	Scaling and performance, part 1	10	Load balancing, horizontal vs vertical, statelessness
19	Scaling and performance, part 2	10	Sharding, replication, CDN, back-pressure
20	Concurrency and parallelism, IO vs CPU bound	11	Goroutines, channels, context, worker pools
21	Containerization and deployment (Docker, K8s, CI/CD)	12	Shipping what you build
22	Automated testing (unit, integration, e2e)	12	Table tests, httptest, testcontainers
23	Message brokers and event streaming (Kafka)	9	Ordering, consumer groups, delivery semantics
24	WebSockets and real-time communication	7	Fan-out, heartbeats, scaling stateful connections
25	OpenAPI specification	4	Contract-first, generated docs and clients
26	AI agents and loop engineering	13	Tool calling, loops, guardrails

Gap list: what interviews ask that the course does not teach in depth

Area	Topics I will add	Part
Networking	DNS resolution, CDN, forward vs reverse proxy, NAT, WebSocket upgrade handshake, HTTP/2 streams	1, 10
Go language	Goroutine scheduler, channels vs mutexes, context , memory model, escape analysis, pprof , generics, error wrapping	11
Databases deep-dive	B-tree vs LSM, query plans, N+1, deadlocks, isolation anomalies, optimistic vs pessimistic locking, SQL vs NoSQL	5
Distributed systems	CAP and PACELC, consistency models, consensus (Raft idea), idempotency, saga, outbox, exactly-once myths	9, 10
System design	URL shortener, rate limiter, chat, news feed, notification system, payment, auction/bidding (your project)	14
Rate limiting and abuse	Token bucket, leaky bucket, sliding window, per-user vs per-IP, distributed limiters	10
Security extras	JWT pitfalls, refresh rotation, password hashing (Argon2id), SSRF, mass assignment, secrets management	3
Operations	Blue/green and canary, feature flags, zero-downtime migrations, backups, incident response, SLO and error budgets	12
Behavioural/situational	"Production is slow", "500s spiking", "DB at 100% CPU", "traffic 10x", debugging playbooks	14

REMEMBER

The part numbers are a plan, not a promise: ask for a different order, or for any topic to be split or merged, and I will adjust. Part 14 (system design and incident scenarios) comes last on purpose: it reuses everything before it.

01 · START HERE

1. The big picture: clients, servers and a request's journey

A **backend** is a program that waits for requests from other programs, does some work (check permissions, read or write a database, call another service) and sends back a response. A **client** is whoever starts the conversation: a browser, a mobile app, `curl`, or another server. A **server** is whoever waits and answers. In HTTP the client always speaks first; the server never sends anything the client did not ask for (the exceptions, SSE and WebSockets, are all started by a client request too).



Figure 1.1 A request's journey. Each box is a place where latency, errors and security decisions live.

Client-server model

WHAT IT IS Two roles. The client initiates, the server listens and responds. They agree on a protocol (HTTP) for the conversation.	WHY WE NEED IT Separation of concerns: the UI can change without touching the data layer, many different clients (web, iOS, partners) can share one backend, and servers can be scaled and secured independently.
USE IT WHEN Almost every networked application: web apps, mobile apps, public APIs, internal microservices.	DON'T USE IT / CAREFUL WHEN When both sides need to push data freely (chat, games) pure request/response is awkward: use WebSockets, SSE or a message broker on top of it. Peer-to-peer apps have no central server.
ADVANTAGES Central place for rules and data (one source of truth); easy to monitor and secure; clients stay thin.	DISADVANTAGES The server is a single point of failure and a bottleneck unless you scale it; every interaction pays network latency.
HOW (in Go / in practice) <code>net.Listen</code> + <code>Accept</code> , or simply <code>http.ListenAndServe</code> . Later parts add layering, config and graceful shutdown.	

Mental model to keep for the whole series: a backend is a pipeline. Bytes arrive, get parsed into a request, pass through a chain of cross-cutting steps (logging, auth, CORS, rate limits), reach a handler that validates input and calls business logic, which talks to storage, and the answer travels back out. Every later chapter is one stage of this pipeline.

2. Statelessness: the first core idea

HTTP has **no memory between requests**. Request 2 does not know request 1 happened. Everything the server needs (who you are, what you want, the format you accept) must travel inside each request, in the URL, headers, cookies or body.

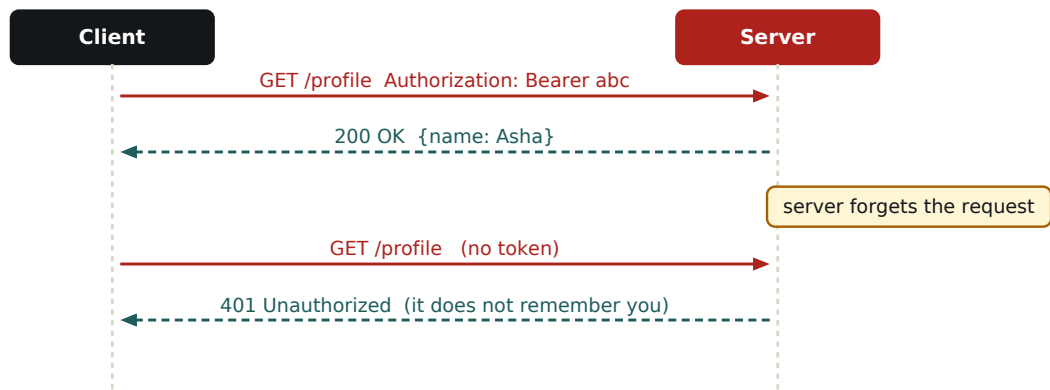


Figure 2.1 Two independent requests. The second fails because the client did not resend its credentials.

Stateless protocol (and how we fake state)

WHAT IT IS The server stores nothing about a client between requests. Continuity is simulated by the client re-sending a token or a session id each time.	WHY WE NEED IT Any server instance can answer any request, so you can add or remove servers freely, and a crash loses no conversation.
USE IT WHEN Always for the HTTP layer. Prefer stateless designs for APIs that must scale horizontally.	DON'T USE IT / CAREFUL WHEN Do not push everything into the client: large per-user state in a JWT bloats every request. Truly stateful needs (WebSocket connections, in-progress uploads, game rooms) need sticky routing or an external store.
ADVANTAGES Simple servers; trivial horizontal scaling and load balancing; failures are isolated; easy caching.	DISADVANTAGES Credentials/context are resent every time (bigger requests); revoking a self-contained token is hard; some features (shopping carts) need an external store anyway.
HOW (in Go / in practice) Keep state outside the process : in Postgres or Redis, or inside a signed token. A Go handler must not rely on a package-level map surviving between requests when you run two replicas.	

Technique	Where the state lives	Good for	Watch out for
Server session + cookie	Server store (Redis/DB); browser holds only a random id	Browser apps; instant revocation	Needs shared store when scaled; CSRF protection needed
Signed token (JWT)	Inside the token the client sends	APIs, mobile, microservices	Cannot be revoked before expiry without extra work; keep payload small
Opaque access token + DB lookup	Server store; client holds random token	Revocable API auth	One lookup per request (cache it)

IN YOUR auctionEngine REPO

auctionEngine keeps both: short-lived access tokens plus refresh tokens (see [docs/decisions/0008-access-and-refresh-tokens.md](#)) and a `internal/session` package. Each request carries its own `Authorization` header, which is exactly the stateless pattern. Redis is treated as a disposable accelerator (ADR 0009), never the only copy of important state.

3. Under HTTP: TCP, TLS and the versions of HTTP

HTTP lives at **layer 7 (application)** of the OSI model. It needs a **reliable, ordered byte stream** underneath, which is what TCP provides (HTTP/3 gets the same guarantees from QUIC over UDP). As a backend engineer you rarely write the lower layers, but you must know what they cost, because that is where latency and many production mysteries come from.

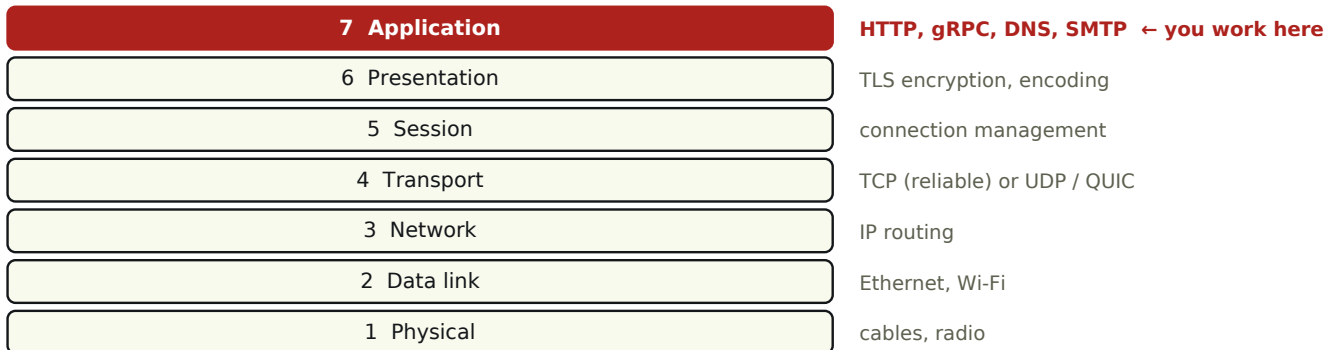


Figure 3.1 The OSI model. HTTP is layer 7; TLS and TCP are what it stands on.

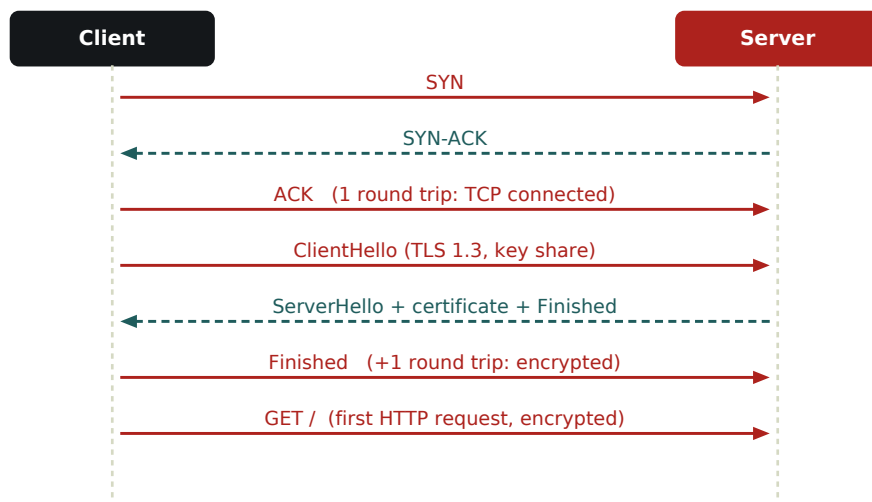


Figure 3.2 What happens before your first byte of HTTP over HTTPS/1.1 or 2: about 2 round trips (TCP, then TLS 1.3).

	TCP	UDP
Delivery	Reliable, ordered, retransmits lost packets	Best effort: may lose, duplicate or reorder
Setup	3-way handshake (1 round trip)	None
Use	HTTP/1.1, HTTP/2, databases, SSH	DNS, video calls, games, QUIC (HTTP/3)
Cost	Handshake latency, head-of-line blocking at the transport layer	You (or QUIC) must handle reliability

The four HTTP versions, and the problem each one solved:

Version	Key idea	Problem it fixed	Remaining weakness
HTTP/1.0 (1996)	One request per TCP connection	First usable version	Open and close a connection for every file: slow
HTTP/1.1 (1997)	Persistent connections (keep-alive), chunked transfer, Host header, better caching	Connection setup cost; virtual hosting	Responses on one connection are in order, so a slow one blocks the next (application-level head-of-line blocking). Browsers open about 6 connections per host to cope

Version	Key idea	Problem it fixed	Remaining weakness
HTTP/2 (2015)	Binary framing, multiplexing many streams on one connection, HPACK header compression	Head-of-line blocking at the HTTP level; verbose headers	Still on TCP: one lost packet stalls all streams (TCP head-of-line blocking)
HTTP/3 (2022)	Runs on QUIC over UDP : per-stream loss recovery, TLS 1.3 built in, 0/1-RTT setup, QPACK	TCP head-of-line blocking; slow handshakes; connection survives network change	UDP may be blocked or throttled by some networks; more CPU per byte

CORRECTION / NUANCE

The video mentions **server push** as an HTTP/2 feature. It existed in the spec but proved hard to use well: Chrome removed support in 2022 and it is effectively dead. Use **103 Early Hints** or `<link rel=preload>` instead. The video says TLS 1.2 is "the current recommended version" (the transcript itself flags that part as uncertain). Today **TLS 1.3** is the recommended version and 1.2 is still acceptable; TLS 1.0 and 1.1 are deprecated (RFC 8996).

AVOID / CAREFUL WHEN

You almost never choose the HTTP version in application code. Go's `net/http` negotiates HTTP/2 automatically over TLS (via ALPN). Choose consciously only at the edge (CDN, load balancer, Nginx), and remember **gRPC requires HTTP/2**.

4. Anatomy of an HTTP message

HTTP/1.1 messages are plain text. A request is: **request line**, **headers**, a **blank line**, then an optional **body**. A response is: **status line**, **headers**, a **blank line**, then the **body**. (HTTP/2 and 3 send the same information in binary frames, but the concepts and header names are identical.)



Figure 4.1 A request message.



Figure 4.2 A response message.

To prove there is no magic, here is a server that speaks HTTP/1.1 by hand over a raw TCP socket (about 70 lines). `net/http` does exactly this, plus the hundreds of edge cases (chunked bodies, timeouts, keep-alive, HTTP/2).

backend-course/code/ch05_http/rawtcp/main.go (the parsing part)

```
func handle(conn net.Conn) {
    defer conn.Close()
    r := bufio.NewReader(conn)

    // 1. Request line: METHOD SP TARGET SP VERSION
    line, err := r.ReadString('\n')
    if err != nil {
        return
    }
    parts := strings.Fields(line)
    if len(parts) != 3 {
        respond(conn, 400, "Bad Request", "malformed request line\n")
        return
    }
    method, target := parts[0], parts[1]

    // 2. Headers: "Key: value" lines until the blank line.
    headers := map[string]string{}
    for {
        h, err := r.ReadString('\n')
        if err != nil {
            return
        }
        h = strings.TrimRight(h, "\r\n")
        if h == "" {
            break // blank line = headers are over, body (if any) starts
        }
        if k, v, ok := strings.Cut(h, ":"); ok {
            headers[strings.ToLower(k)] = strings.TrimSpace(v)
        }
    }

    // 3. Route by hand.
    switch {
    case method == "GET" && target == "/hello":
        respond(conn, 200, "OK", "hello from raw TCP, you are "+headers["user-agent"]+"\n")
    case target == "/hello":
        respond(conn, 405, "Method Not Allowed", "use GET\n")
    default:
        respond(conn, 404, "Not Found", fmt.Sprintf("no route for %s\n", target))
    }
}
```

...and the response writer

```
func respond(conn net.Conn, code int, reason, body string) {
    // Status line, headers, blank line, body.
    // Content-Length tells the client where the body ends.
    fmt.Fprintf(conn, "HTTP/1.1 %d %s\r\n", code, reason)
    fmt.Fprint(conn, "Content-Type: text/plain; charset=utf-8\r\n")
    fmt.Fprintf(conn, "Content-Length: %d\r\n", len(body))
    fmt.Fprint(conn, "Connection: close\r\n\r\n")
    fmt.Fprint(conn, body)
}
```

Real output from running the program

```
$ curl -i localhost:8081/hello -H 'User-Agent: demo'
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 33
Connection: close

hello from raw TCP, you are demo

$ curl -si -X POST localhost:8081/hello | head -1
HTTP/1.1 405 Method Not Allowed
```

REMEMBER

Content-Length (or chunked encoding) is how the receiver knows where the body ends on a connection that stays open. Get it wrong and the client hangs waiting for bytes that never come, or reads part of the next response.

5. Headers: the metadata layer

Headers are **Key: value** pairs that describe the message, not its content. **Why not put this in the URL or the body?** The video's parcel analogy is right: the courier needs the address written **on the outside** so every handler along the route (proxies, load balancers, caches, browsers) can act without opening the box. Headers let infrastructure that never understands your JSON still route, cache, compress, authenticate and log your traffic.

Category	Examples	Purpose
Request	User-Agent, Authorization, Accept, Accept-Language, Accept-Encoding, Origin, Host, Cookie, If-None-Match	Tell the server who is asking and what the client prefers or can handle
Response	Set-Cookie, Location, Retry-After, WWW-Authenticate, Allow, Server	Server instructions and extra info about the answer
General (both)	Date, Cache-Control, Connection, Via	Metadata about the message or connection itself
Representation (both)	Content-Type, Content-Length, Content-Encoding, Content-Language, ETag, Last-Modified	Describe the body: its type, size, encoding, version
Security	Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, cookie flags	Make browsers refuse risky behaviour
CORS	Access-Control-Allow-Origin, -Allow-Methods, -Allow-Headers, -Max-Age, -Allow-Credentials	Cross-origin permission (section 7)
Proxy/infra	X-Forwarded-For, X-Forwarded-Proto, X-Request-ID, Forwarded	Preserve client info across proxies; trace a request through systems

Two ideas from the video: headers are **extensible** (you can invent `X-My-Header` and nothing in the protocol breaks) and act as a **remote control** (the client steers the server with `Accept`, `Authorization`, `If-None-Match`; the server steers the client with `Cache-Control`, `Set-Cookie`, `Location`).

Security header	What it stops	Typical API value
Strict-Transport-Security	Protocol downgrade and cookie theft on plain HTTP	<code>max-age=31536000; includeSubDomains</code>
Content-Security-Policy	XSS: restricts where scripts, styles, images may load from	API: <code>default-src 'none'; frame-ancestors 'none'</code>
X-Content-Type-Options	MIME sniffing (a JSON file being run as a script)	<code>nosniff</code>
X-Frame-Options	Clickjacking (your page inside an attacker's iframe). Modern form: CSP <code>frame-ancestors</code>	<code>DENY</code>
Referrer-Policy	URLs with IDs leaking to other sites	<code>no-referrer</code>
Set-Cookie flags	<code>HttpOnly</code> : JS cannot read it. <code>Secure</code> : HTTPS only. <code>SameSite</code> : limits cross-site sending (CSRF)	<code>HttpOnly; Secure; SameSite=Lax</code>

Here is the real middleware from your own repo that sets them. It is a good example of a **middleware**: a function that wraps a handler, does something before and/or after, and calls `next`:

```

auctionEngine/internal/middleware/middleware.go

func SecurityHeaders(hsts bool) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            h := w.Header()
            h.Set("X-Content-Type-Options", "nosniff")
            h.Set("X-Frame-Options", "DENY")
            h.Set("Content-Security-Policy", "default-src 'none'; frame-ancestors 'none'")
            h.Set("Referrer-Policy", "no-referrer")
            if hsts {
                h.Set("Strict-Transport-Security", "max-age=31536000; includeSubDomains")
            }
            next.ServeHTTP(w, r)
        })
    }
}

```

COMMON MISTAKE

Header names are case-insensitive (`content-type` equals `Content-Type`). Go canonicalizes them: use `r.Header.Get("content-type")`, but read the map directly and you must use canonical form.

Never trust `X-Forwarded-For` or `X-Forwarded-Proto` unless the request really came from your own proxy; anyone can send them. Never put secrets in a URL: URLs end up in logs, browser history and `Referer` headers. Put them in a header.

6. Methods: expressing intent

The method says **what you intend to do** with the resource at the URL. That matters because proxies, caches, browsers and retry logic treat methods differently without ever reading your code.

Method	Intent	Has body?	Safe?	Idempotent?	Cachable?	Success code
GET	Read a resource	No	Yes	Yes	Yes	200
HEAD	Like GET, headers only (check existence, size, ETag)	No	Yes	Yes	Yes	200
POST	Create / run an action	Yes	No	No	Rarely	201 (+ Location), 200, 202
PUT	Replace the whole resource at this URL	Yes	No	Yes	No	200, 204 (201 if created)
PATCH	Partially update	Yes	No	Not guaranteed	No	200, 204
DELETE	Remove	Optional	No	Yes	No	204 (or 200)
OPTIONS	Ask what is allowed (CORS pre-flight)	No	Yes	Yes	No	204

Safe means "does not change server state" (so a crawler or prefetch may call it freely). **Idempotent** means "calling it N times leaves the server in the same state as calling it once".

Idempotency

WHAT IT IS A property of an operation: repeating the same request has the same effect on server state as sending it once.	WHY WE NEED IT Networks fail. If a client times out it does not know whether the server processed the request. Idempotent operations can be retried blindly; non-idempotent ones risk a duplicate (two charges, two bids, two emails).
USE IT WHEN Design every endpoint that may be retried (mobile apps, background workers, API gateways, payment and bidding calls) so a retry is harmless.	DON'T USE IT / CAREFUL WHEN Idempotent does not mean "same response": the 2nd DELETE returns 404 but the state (gone) is identical. Do not assume PATCH is idempotent ("add 10 to balance" is not).
ADVANTAGES Safe retries, simple client code, resilient systems, can be built on automatic retry middleware.	DISADVANTAGES Extra work for POST: you need keys, storage with TTL, and care with concurrent duplicates.
HOW (in Go / in practice) Use PUT with a client-chosen id, or POST with an Idempotency-Key header: the server stores the first result and replays it for repeats (code below).	

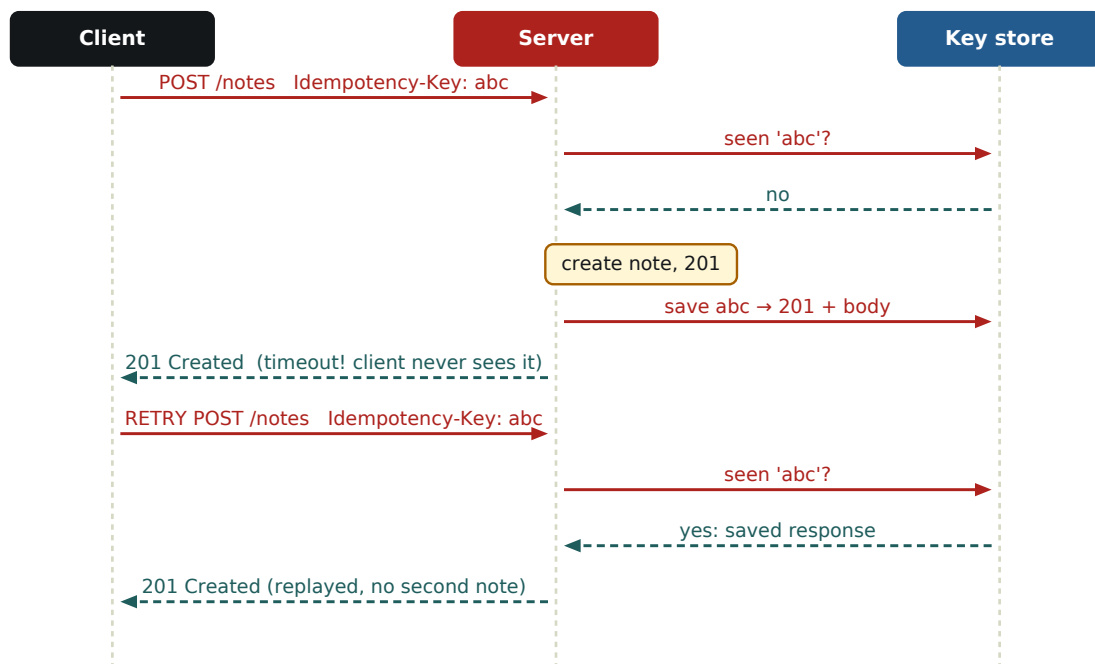


Figure 6.1 Idempotency key. The retry returns the stored answer; only one note exists.

code/ch05_http/server/idempotency.go (tested)

```

func Idempotent(next http.Handler) http.Handler {
    type saved struct {
        status int
        header http.Header
        body []byte
    }
    var (
        mu sync.Mutex
        seen = map[string]*saved{}
    )
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        key := r.Header.Get("Idempotency-Key")
        if key == "" {
            next.ServeHTTP(w, r)
        }
    })
}

```

```

        return
    }
    mu.Lock()
    defer mu.Unlock() // simple version: serialises keyed requests (also blocks the race)
    if s, ok := seen[key]; ok {
        for k, v := range s.header {
            w.Header()[k] = v
        }
        w.Header().Set("Idempotent-Replayed", "true")
        w.WriteHeader(s.status)
        w.Write(s.body)
        return
    }
    rec := &capture{ResponseWriter: w, status: http.StatusOK}
    next.ServeHTTP(rec, r)
    if rec.status < 500 { // never cache server errors: let the client retry
        seen[key] = &saved{rec.status, rec.Header().Clone(), rec.buf.Bytes()}
    }
}
})
}

```

PUT vs PATCH in the sample server: **PUT** demands a complete representation and replaces it. **PATCH** uses pointer fields so "field absent" is different from "field set to empty":

```

code/ch05_http/server/handlers.go

// PATCH changes only the fields that were sent (pointer == "was it present?").
func (a *api) patch(w http.ResponseWriter, r *http.Request) {
    id, ok := a.id(w, r)
    if !ok {
        return
    }
    var in struct {
        Text *string `json:"text"`
    }
    if !decode(w, r, &in) {
        return
    }
    a.s.mu.Lock()
    defer a.s.mu.Unlock()
    n, found := a.s.notes[id]
    if !found {
        writeErr(w, http.StatusNotFound, "note not found")
        return
    }
    if in.Text != nil {
        n.Text = *in.Text
    }
    n.UpdatedAt = time.Now().UTC().Truncate(time.Second)
    a.s.notes[id] = n
    writeJSON(w, http.StatusOK, n)
}

```

CORRECTION / NUANCE

The video says: "always use PATCH unless you have a specific reason to use PUT". That is a fine rule of thumb for update endpoints. Add that PUT is also correct when the **client chooses the resource id** (create-or-replace, **PUT** `/users/me/avatar`), and that many large APIs deliberately offer only POST and PATCH.

The video's CORS section says a request is pre-flighted if the method is "PUT or DELETE". More exactly: **anything other than GET, HEAD or POST**, which also includes PATCH.

INTERVIEW ANGLE

"Is DELETE idempotent if the second call returns 404?" **Yes**. Idempotency is about the resulting state of the server, not about getting identical status codes.

7. CORS, the same-origin policy and pre-flight

An **origin** is **scheme + host + port**. `http://localhost:5173` and `http://localhost:3000` are different origins; so are `https://a.com` and `https://api.a.com`. The browser's **same-origin policy** lets JavaScript on one origin send requests anywhere but blocks it from reading responses from another origin, unless that origin opts in with CORS headers.

CORRECTION / NUANCE

CORS is enforced by the browser, not the server. The server only states a policy. `curl`, Postman, mobile apps and other servers ignore CORS entirely, so CORS is **not** protection for your API: you still need authentication. Also, with a simple request the request is still sent and executed; CORS only hides the response from the page. This is why CSRF is a separate problem.

A "CORS error" in the console almost always means the **server response is missing a header** (or the pre-flight failed), not that the frontend code is wrong. Fix it on the server.

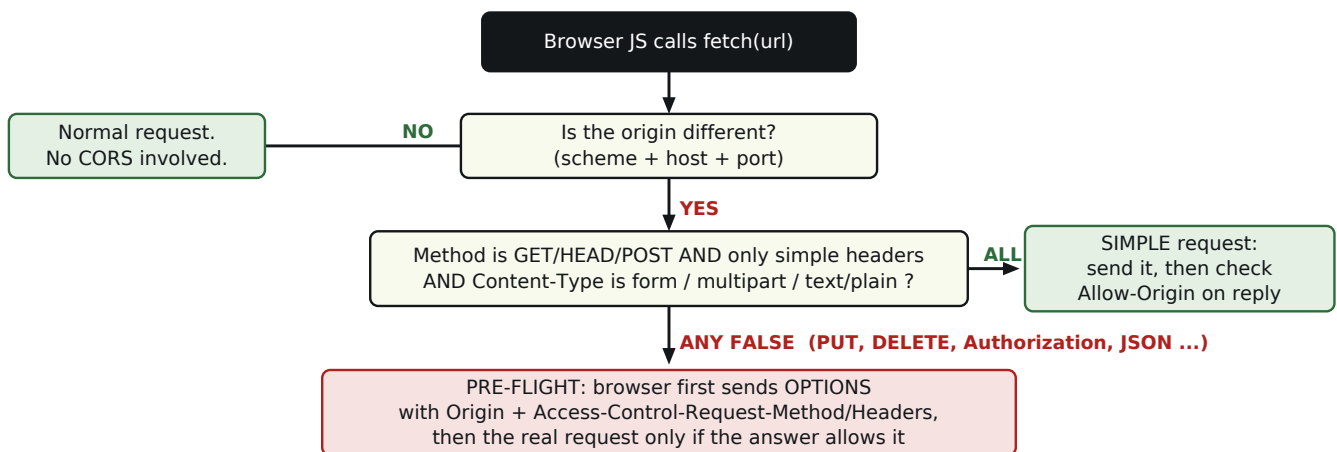


Figure 7.1 How a browser decides between a simple request and a pre-flight request.



Figure 7.2 Simple request. If the Allow-Origin header is missing or different, the browser hides the response (CORS error).

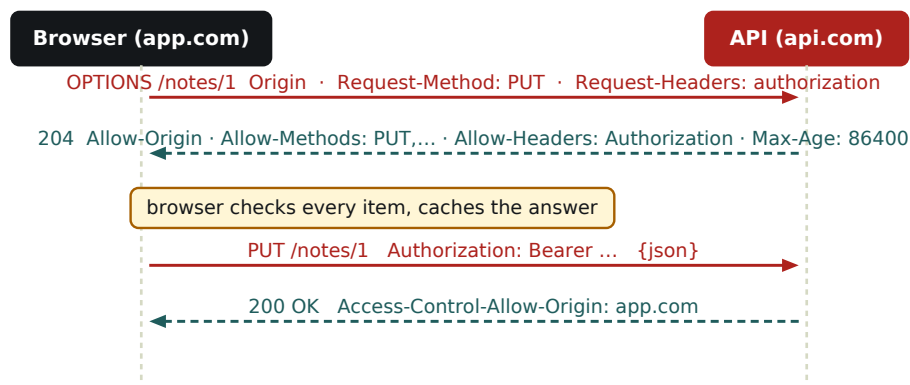


Figure 7.3 Pre-flight. Two round trips for the first call. Max-Age lets the browser skip the first one next time.

Pre-flight triggered when the request is cross-origin AND at least one is true	Example
Method is not GET, HEAD or POST	PUT, PATCH, DELETE

Pre-flight triggered when the request is cross-origin AND at least one is true	Example
It carries a non-safelisted header	Authorization, X-Request-ID, Idempotency-Key
Content-Type is not form-urlencoded, multipart/form-data or text/plain	application/json (so most JSON APIs always pre-flight)

code/ch05_http/server/middleware.go

```

func CORS(allowed []string) func(http.Handler) http.Handler {
    set := make(map[string]bool, len(allowed))
    for _, o := range allowed {
        set[o] = true
    }
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            origin := r.Header.Get("Origin")
            w.Header().Add("Vary", "Origin")

            // Not a cross-origin browser request, or origin not allowed:
            // do nothing. The browser will block the response for us.
            if origin == "" || !set[origin] {
                next.ServeHTTP(w, r)
                return
            }
            w.Header().Set("Access-Control-Allow-Origin", origin)

            isPreflight := r.Method == http.MethodOptions &&
                r.Header.Get("Access-Control-Request-Method") != ""
            if isPreflight {
                h := w.Header()
                h.Set("Access-Control-Allow-Methods", "GET, POST, PUT, PATCH, DELETE")
                h.Set("Access-Control-Allow-Headers", "Content-Type, Authorization")
                h.Set("Access-Control-Max-Age", "86400") // cache the answer for 24h
                w.WriteHeader(http.StatusNoContent)    // 204: nothing to say, only headers
                return
            }
            next.ServeHTTP(w, r)
        })
    }
}

```

Your production version adds **Expose-Headers**, an **Idempotency-Key** allowance and a 403 for pre-flights from unknown origins:

auctionEngine/internal/middleware/middleware.go

```

const (
    methods = "GET, POST, PUT, PATCH, DELETE, OPTIONS"
    headers = "Authorization, Content-Type, Idempotency-Key, X-Request-ID"
    maxAge  = 600 // seconds a browser may cache the preflight answer

```

CORS

WHAT IT IS A set of Access-Control-* response headers by which a server tells browsers which other origins may read its responses and with which methods and headers.	WHY WE NEED IT Without it any website you visit could read your logged-in data from other sites. It is the safe default that lets you opt in selectively.
USE IT WHEN Whenever a browser frontend on one origin calls an API on another (SPA on <code>app.example.com</code> , API on <code>api.example.com</code> ; Vite dev server on <code>:5173</code> calling <code>:8080</code>).	DON'T USE IT / CAREFUL WHEN Do not use <code>*</code> for authenticated APIs. Do not echo back any Origin header blindly (that is the same as <code>*</code>). Not needed if the frontend and API are served from the same origin (put both behind one reverse proxy).
ADVANTAGES Fine-grained, standard, cached by browsers via Max-Age .	DISADVANTAGES Extra round trip for pre-flighted calls; easy to misconfigure; confusing errors; says nothing about non-browser clients.
HOW (in Go / in practice) Use an allow-list middleware as above, add Vary: Origin , and put it before authentication, because pre-flight requests carry no credentials.	

COMMON MISTAKE

Access-Control-Allow-Origin: * together with **Allow-Credentials: true** is **rejected by browsers**. With cookies you must echo one specific origin.

Forgetting **Vary: Origin** lets a shared cache serve a response with the wrong **Allow-Origin** to another site.

Only handling CORS on success: error responses (401, 404, 500) also need the headers or the frontend sees a CORS error instead of the real error.

Browsers cap **Max-Age**: Chromium about 2 hours, Firefox 24 hours, whatever you send. In the video's demo it was set to 0 only for testing.

Auth middleware placed before CORS rejects the pre-flight with 401, and the browser reports a CORS failure.

FROM THE VIDEO

The video demonstrates this with **Burp Suite** (an intercepting proxy for security testing). You can see the same thing for free in the browser DevTools **Network** tab, or with `curl -i -X OPTIONS ... -H 'Origin: ...' -H 'Access-Control-Request-Method: PUT'` against **your own** servers. Only intercept traffic of apps you own or are authorised to test.

8. Status codes: the universal result language

A status code lets a client decide what to do **without parsing the body**: retry, log the user out, show a form error, or stop. Because the codes are standard, a Go server, a Rust server and a Python server look the same to every client and proxy.

Range	Meaning	Client's reaction
1xx	Informational	Keep going: 100 Continue , 101 Switching Protocols (WebSocket upgrade), 103 Early Hints
2xx	Success	Use the result
3xx	Redirection	Look elsewhere or use your cache
4xx	Client error: the request is wrong	Do not retry unchanged (except 408/429); fix the request
5xx	Server error: we failed	Safe to retry later with back-off (if idempotent)

Code	Name	Use it when	Go constant
200	OK	Successful GET/PUT/PATCH with a body	StatusOK
201	Created	POST created something; add a Location header	StatusCreated

Code	Name	Use it when	Go constant
202	Accepted	Work queued for later (background job); not done yet	<code>StatusAccepted</code>
204	No Content	Success with nothing to return (DELETE, pre-flight)	<code>StatusNoContent</code>
301	Moved Permanently	URL changed forever (browsers and search engines cache it)	<code>StatusMovedPermanently</code>
302	Found	Temporary redirect (campaign, login bounce)	<code>StatusFound</code>
304	Not Modified	Conditional GET matched: use your cached copy, no body	<code>StatusNotModified</code>
307 / 308	Temp / Perm redirect	Like 302/301 but the method and body are preserved (use for POST redirects)	<code>StatusTemporaryRedirect</code>
400	Bad Request	Malformed or invalid input (bad JSON, wrong type, failed validation)	<code>StatusBadRequest</code>
401	Unauthorized	Not authenticated: no, expired or invalid credentials. Send <code>WWW-Authenticate</code>	<code>StatusUnauthorized</code>
403	Forbidden	Authenticated but not allowed (user A deleting user B's data)	<code>StatusForbidden</code>
404	Not Found	No such resource (or you hide that it exists)	<code>StatusNotFound</code>
405	Method Not Allowed	Path exists, method does not. Send <code>Allow</code> . Go's mux does it for you	<code>StatusMethodNotAllowed</code>
406	Not Acceptable	Cannot produce any format the <code>Accept</code> header allows	<code>StatusNotAcceptable</code>
409	Conflict	State conflict: duplicate name, edit collision, outbid	<code>StatusConflict</code>
412	Precondition Failed	<code>If-Match</code> did not match (optimistic locking)	<code>StatusPreconditionFailed</code>
413	Content Too Large	Body over your limit	<code>StatusRequestEntityTooLarge</code>
415	Unsupported Media Type	You only accept JSON and got something else	<code>StatusUnsupportedMediaType</code>
422	Unprocessable Content	Syntax fine, meaning invalid (many APIs just use 400)	<code>StatusUnprocessableEntity</code>
429	Too Many Requests	Rate limited. Send <code>Retry-After</code>	<code>StatusTooManyRequests</code>
500	Internal Server Error	Unhandled failure. Log details, return a generic message	<code>StatusInternalServerError</code>
502	Bad Gateway	A proxy got an invalid reply from your app	<code>StatusBadGateway</code>
503	Service Unavailable	Overloaded, deploying or draining. Send <code>Retry-After</code>	<code>StatusServiceUnavailable</code>
504	Gateway Timeout	A proxy gave up waiting for your app	<code>StatusGatewayTimeout</code>

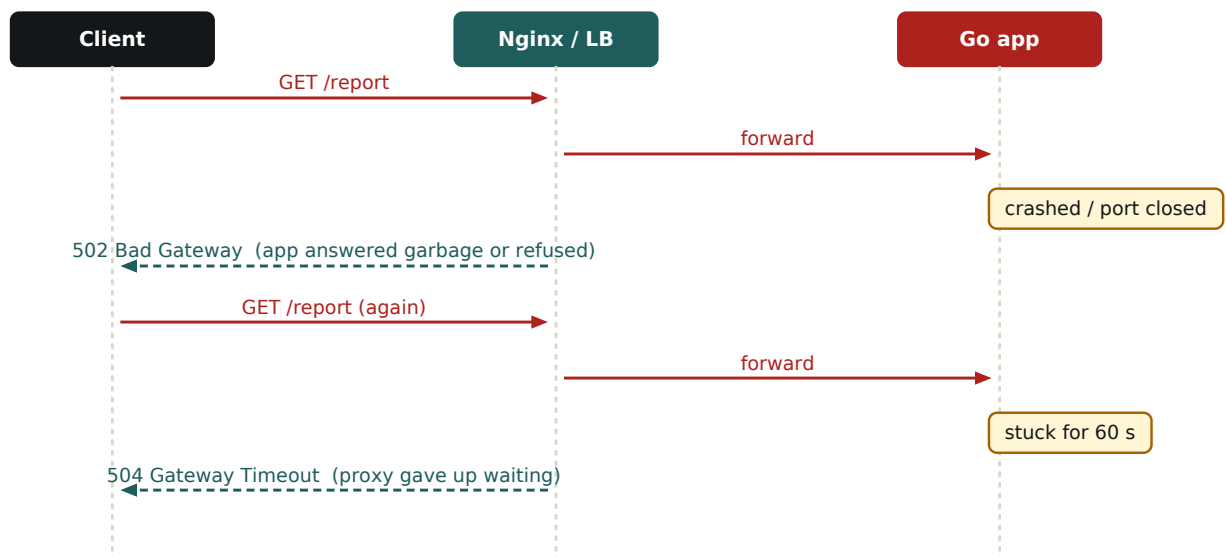


Figure 8.1 502 vs 504. Your code rarely returns these; proxies do. Seeing them tells you the problem is the app or its connection.

CORRECTION / NUANCE

401 vs 403. 401's name says "unauthorized" but it means **unauthenticated** ("I don't know who you are"). 403 means "I know who you are, and no". The video explains this correctly; just memorise it as authn vs authz.

Hiding existence: for private resources return **404** instead of 403 so you do not reveal that the id exists.

Validation errors: the video uses 400. Both 400 and 422 are defensible; be consistent and always return a machine-readable body (`{"error": "invalid input", "fields": {"email": "is required"}}`, which is what your `httpx` package does).

301 vs 302 vs 308: old browsers turned a redirected POST into a GET for 301/302. Use 307/308 when the method must be preserved.

The consistent-errors helper from your own repo, the pattern every handler should use so errors look the same everywhere:

```

auctionEngine/internal/httpx/httpx.go

type ErrorResponse struct {
    Error string `json:"error"`
    Fields map[string]string `json:"fields,omitempty"`
}

// WriteJSON writes v as JSON with the given status code.
func WriteJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)

    if err := json.NewEncoder(w).Encode(v); err != nil {
        slog.Error("failed to encode response", "error", err)
    }
}

// Error writes {"error": message}.
func Error(w http.ResponseWriter, status int, message string) {
    WriteJSON(w, status, ErrorResponse{Error: message})
}

// ValidationError writes a 400 listing what is wrong with each field.
func ValidationError(w http.ResponseWriter, problems *validation.Error) {
    ...
}

```

COMMON MISTAKE

Returning **200 with** `{"success": false}` breaks every client, cache, monitor and retry policy. Use the real status code.

Returning 500 for client mistakes (bad input) pollutes your error-rate alerts. Returning 200 for server crashes hides outages.

Leaking internals in 500 bodies (stack traces, SQL) is an information-disclosure bug: log it, return a generic message.

9. HTTP caching: reuse what has not changed

Caching stores a copy of a response so the next request can skip work. Two mechanisms work together:

- **Freshness** (`Cache-Control: max-age=N`): for N seconds the client reuses its copy **without asking the server at all**.
- **Validation** (`ETag` + `If-None-Match`, or `Last-Modified` + `If-Modified-Since`): after the copy goes stale the client asks "has it changed?". If not, the server answers `304 Not Modified` with **no body**, saving bandwidth.

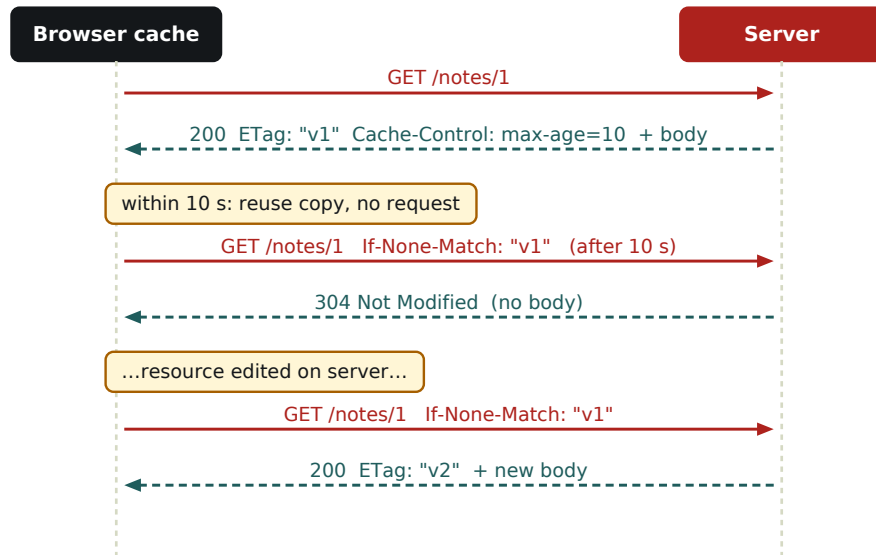


Figure 9.1 The caching conversation from the video's demo (the video used 10 s max-age and numeric ETags).

code/ch05_http/server/handlers.go

```
// GET is safe + idempotent + cacheable. Conditional GET returns 304.
func (a *api) get(w http.ResponseWriter, r *http.Request) {
    id, ok := a.id(w, r)
    if !ok {
        return
    }
    a.s.mu.RLock()
    n, found := a.s.notes[id]
    a.s.mu.RUnlock()
    if !found {
        writeErr(w, http.StatusNotFound, "note not found") // 404
        return
    }
    tag := etag(n)
    h := w.Header()
    h.Set("ETag", tag)
    h.Set("Last-Modified", n.UpdatedAt.UTC().Format(http.TimeFormat))
    h.Set("Cache-Control", "max-age=10") // fresh for 10s, then revalidate

    // RFC 9110: If-None-Match wins over If-Modified-Since.
    if inm := r.Header.Get("If-None-Match"); inm != "" {
        if inm == tag || inm == "*" {
            w.WriteHeader(http.StatusNotModified) // 304: no body, use your cache
            return
        }
    }
    if ims := r.Header.Get("If-Modified-Since"); ims != "" {
        if t, err := http.ParseTime(ims); err == nil && !n.UpdatedAt.After(t) {
            w.WriteHeader(http.StatusNotModified)
            return
        }
    }
    negotiate(w, r, n)
}
```

Real output from the server in this PDF's repo

```
$ curl -si localhost:8082/notes/1 | grep -iE "^HTTP|etag|cache-control|last-mod"
HTTP/1.1 200 OK
Cache-Control: max-age=10
Etag: "1aa11812ea0bfb60"
Last-Modified: Sat, 03 Oct 2026 16:23:23 GMT

$ curl -si -H 'If-None-Match: "1aa11812ea0bfb60"' localhost:8082/notes/1 | head -1
HTTP/1.1 304 Not Modified
```

Directive	Meaning	Typical use
<code>max-age=N</code>	Fresh for N seconds in any cache	Static assets, rarely changing data
<code>s-maxage=N</code>	Like max-age but only for shared caches (CDN)	CDN caches longer than browsers
<code>no-cache</code>	Stored, but must revalidate before every use (name is misleading)	Data that must be current but can be 304
<code>no-store</code>	Never store anywhere	Banking pages, tokens, personal data
<code>private / public</code>	Only the browser may cache / any cache may cache	<code>private</code> for user-specific responses
<code>immutable</code>	Will never change at this URL	Fingerprinted files like <code>app.3f9a1c.js</code>
<code>stale-while-revalidate=N</code>	Serve stale instantly while refreshing in the background	Fast UI for slightly stale data
<code>Vary: Accept-Encoding</code>	The cache key also includes these request headers	Required whenever content negotiation changes the body

HTTP caching (ETag, Cache-Control, Last-Modified)

WHAT IT IS Browsers, proxies and CDNs keep copies of responses and reuse them when the rules in the headers allow, or revalidate cheaply with a conditional request.	WHY WE NEED IT Fewer bytes over the network, lower latency (zero round trips while fresh), lower server and database load.
USE IT WHEN Public or rarely changing data: static files, product catalogues, images, configuration, public API reads.	DON'T USE IT / CAREFUL WHEN User-specific or sensitive data (use <code>private</code> or <code>no-store</code>). Data that must be exact in real time (live bid price). Anything where a stale copy causes harm.
ADVANTAGES Free and built into every browser and CDN; huge savings; works for any client that speaks HTTP.	DISADVANTAGES Staleness is hard to fix (you cannot push-invalidate a browser cache); easy to forget to change the ETag; the wrong <code>Vary</code> serves one user another's data.
HOW (in Go / in practice) Generate a strong ETag from the representation (hash) or a version number, honour <code>If-None-Match</code> before doing expensive work, always set an explicit <code>Cache-Control</code> .	

CORRECTION / NUANCE

The video says client-side libraries (React Query/TanStack Query) are "better" than HTTP caching. They solve a different layer: they give the app control of when to refetch, while HTTP caching is understood by **every** intermediary and costs the client nothing. Real systems use both, plus server-side caching (Redis, cache-aside) which is Part 6. Without any `Cache-Control`, browsers may still cache using a **heuristic** based on `Last-Modified`. Always be explicit. `ETag` is usually a hash of the response, but a version column or `updated_at` works too. Weak ETags (`W/"..."`) mean "semantically the same".

INTERVIEW ANGLE

"Users see old data after a deploy." → Check **Cache-Control** on the HTML and JS. Fix: no-cache on the HTML entry point, long **max-age** + **immutable** on content-hashed asset filenames, so a new deploy changes the URLs.

10. Content negotiation and compression

The client states what it can handle and prefers; the server picks the best match and **says what it chose**.

Three axes:

Axis	Request header	Response header	Example
Media type	Accept: <code>application/json</code>	Content-Type	JSON vs XML vs HTML
Language	Accept-Language: <code>es-ES,es;q=0.9</code>	Content-Language	Hola vs Hello
Encoding	Accept-Encoding: <code>gzip, br</code>	Content-Encoding	Compress the body

code/ch05_http/server/handlers.go

```
// negotiate picks JSON or XML from the Accept header.
func negotiate(w http.ResponseWriter, r *http.Request, v Note) {
    accept := r.Header.Get("Accept")
    w.Header().Add("Vary", "Accept")
    switch {
    case strings.Contains(accept, "application/xml"), strings.Contains(accept, "text/xml"):
        w.Header().Set("Content-Type", "application/xml")
        w.WriteHeader(http.StatusOK)
        _ = xml.NewEncoder(w).Encode(v)
    case accept == "", strings.Contains(accept, "application/json"), strings.Contains(accept, "*/"):
        writeJSON(w, http.StatusOK, v)
    default:
        writeErr(w, http.StatusNotAcceptable, "supported: application/json, application/xml") // 406
    }
}
```

Real output

```
$ curl -s -H 'Accept-Language: es' localhost:8082/hello
{"lang":"es","message":"Hola"}
$ curl -s -H 'Accept-Language: hi' localhost:8082/hello
{"lang":"hi","message":"Namaste"}
$ curl -si localhost:8082/notes/1 -H 'Accept: image/png' | head -1
HTTP/1.1 406 Not Acceptable
```

Content negotiation

WHAT IT IS Client and server agree on the representation of a resource (format, language, encoding) using Accept* request headers and Content-* response headers.	WHY WE NEED IT One URL can serve many clients: mobile app wants compact JSON, a legacy partner wants XML, users in Spain want Spanish text.
USE IT WHEN Public APIs with several formats, internationalised content, supporting compression.	DON'T USE IT / CAREFUL WHEN Most internal JSON APIs: just return JSON and answer 406/415 for anything else. Do not build five formats you will never test.
ADVANTAGES Flexible without extra URLs; standard; enables compression.	DISADVANTAGES More code paths and tests; caches must key on Vary ; clients often send sloppy Accept headers (/*/*).
HOW (in Go / in practice) Parse Accept (with q-values in real code), set Vary: Accept , answer 406 when nothing matches.	

Compression

The server compresses the body (gzip, or Brotli **br**, which compresses text better but costs more CPU) and the client decompresses it transparently. Measured on this PDF's sample server for the same 50,000-row JSON response:



Figure 10.1 2,638,891 bytes → 131,744 bytes (about 20× smaller). Real numbers from ``curl -s /big | wc -c``.

code/ch05_http/server/middleware.go

```
// Gzip compresses the response when the client advertises support through
// Accept-Encoding (content negotiation for encoding).
func Gzip(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Add("Vary", "Accept-Encoding")
        if !strings.Contains(r.Header.Get("Accept-Encoding"), "gzip") {
            next.ServeHTTP(w, r)
            return
        }
        w.Header().Set("Content-Encoding", "gzip")
        w.Header().Del("Content-Length") // length changes after compression
        gz := gzip.NewWriter(w)
        defer gz.Close()
        next.ServeHTTP(&gzipWriter{ResponseWriter: w, zw: gz}, r)
    })
}

type gzipWriter struct {
    http.ResponseWriter
    zw *gzip.Writer
}

func (g *gzipWriter) Write(b []byte) (int, error) { return g.zw.Write(b) }
```

AVOID / CAREFUL WHEN

Do **not** compress already-compressed data (JPEG, PNG, MP4, ZIP): wasted CPU, sometimes larger output. Skip tiny responses (under about 1 KB).

Compressing responses that mix **secrets with attacker-controlled input** over HTTPS can enable the **BREACH** attack; keep CSRF tokens out of compressible, reflected pages or mask them.

In production, usually let the reverse proxy or CDN compress, not your Go code. Go's client transport automatically sends **Accept-Encoding: gzip** and decompresses, unless you set that header yourself.

11. Persistent connections, large requests and streaming

Keep-alive (persistent connections)

Opening a connection costs one TCP round trip, plus one or two for TLS. HTTP/1.0 paid that for **every** request. HTTP/1.1 keeps the connection open and reuses it by default (**Connection: keep-alive** is the default; **Connection: close** ends it). HTTP/2 goes further and multiplexes many requests on one connection.

Persistent connections

WHAT IT IS One TCP (and TLS) connection carries many request-response pairs in sequence until either side closes it or it sits idle too long.	WHY WE NEED IT Skips repeated TCP and TLS handshakes; avoids exhausting ports and file descriptors; far lower latency for sequences of calls.
USE IT WHEN Always, by default. Especially for service-to-service calls and anything with many small requests.	DON'T USE IT / CAREFUL WHEN Idle connections hold memory and file descriptors on the server: set <code>IdleTimeout</code> . Behind load balancers, align your idle timeout with theirs to avoid resets.
ADVANTAGES Big latency and CPU savings with no code.	DISADVANTAGES Connections tie up server resources; a single slow response can block later ones on that HTTP/1.1 connection; stale reused connections can fail once.
HOW (in Go / in practice) Server: set <code>IdleTimeout</code> . Client: reuse one shared <code>http.Client</code> (never create one per call), close response bodies, tune <code>MaxIdleConnsPerHost</code> (default 2).	

code/ch05_http/client/client.go

```
func newClient() *http.Client {
    dialer := &net.Dialer{Timeout: 3 * time.Second, KeepAlive: 30 * time.Second}
    tr := &http.Transport{
        DialContext:    dialer.DialContext,
        TLSHandshakeTimeout: 5 * time.Second,
        ResponseHeaderTimeout: 10 * time.Second,
        MaxIdleConns:    100,
        MaxIdleConnsPerHost: 20, // default is 2: too low for a service calling one host a lot
        IdleConnTimeout: 90 * time.Second,
    }
    return &http.Client{Transport: tr, Timeout: 15 * time.Second} // overall cap per request
}

func getJSON(ctx context.Context, c *http.Client, url string, out any) error {
    req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
    if err != nil {
        return err
    }
    req.Header.Set("Accept", "application/json")
    resp, err := c.Do(req)
    if err != nil {
        return err
    }
    defer resp.Body.Close()
    if resp.StatusCode != http.StatusOK {
        io.Copy(io.Discard, io.LimitReader(resp.Body, 1<<20)) // drain so the connection is reusable
        return fmt.Errorf("unexpected status %s", resp.Status)
    }
    return json.NewDecoder(resp.Body).Decode(out)
}
```

...and the test that proves 5 requests use 1 TCP connection

```
// Five sequential requests must share ONE TCP connection (HTTP/1.1 keep-alive).
func TestKeepAliveReusesConnection(t *testing.T) {
    var conns atomic.Int32
    srv := httptest.NewUnstartedServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Write([]byte(`{"ok":"yes"}`))
    }))
    srv.Config.ConnState = func(_ net.Conn, s http.ConnState) {
        if s == http.StateNew {
            conns.Add(1)
        }
    }
    srv.Start()
    defer srv.Close()
```

```

c := newClient()
for i := 0; i < 5; i++ {
    var out map[string]string
    if err := getJSON(context.Background(), c, srv.URL, &out); err != nil {
        t.Fatal(err)
    }
}
if got := conns.Load(); got != 1 {
    t.Fatalf("opened %d TCP connections for 5 requests, want 1", got)
}
}

```

COMMON MISTAKE

`http.Get(url)` uses `http.DefaultClient`, which has **no timeout**: one slow upstream can hang goroutines forever and cascade into an outage.
 Forgetting `resp.Body.Close()` leaks connections. Not reading the body to the end prevents reuse. The cure is `defer resp.Body.Close()` and draining on error paths.
 Creating a new `http.Client` per request throws away the connection pool.

Large requests: multipart/form-data

A file does not fit a JSON body (it would need Base64, which is 33% bigger and must sit in memory).

multipart/form-data splits the body into parts separated by a **boundary** string declared in **Content-Type**. Each part has its own headers (field name, filename, type):

A multipart request on the wire

```

POST /upload HTTP/1.1
Content-Type: multipart/form-data; boundary=---X9f2
Content-Length: 5209

-----X9f2
Content-Disposition: form-data; name="photo"; filename="car.jpg"
Content-Type: image/jpeg

<5000 bytes of binary data>
-----X9f2--

```

code/ch05_http/server/handlers.go (streams parts, never loads the whole file)

```

// upload reads a multipart/form-data body as a STREAM (never the whole file
// in memory) and reports how many bytes each file part contained.
func upload(w http.ResponseWriter, r *http.Request) {
    r.Body = http.MaxBytesReader(w, r.Body, 2<<30) // 2 GB hard ceiling
    mr, err := r.MultipartReader()
    if err != nil {
        writeErr(w, http.StatusBadRequest, "expected multipart/form-data: "+err.Error())
        return
    }
    type result struct {
        Field string `json:"field"`
        File  string `json:"filename"`
        Bytes int64  `json:"bytes"`
    }
    var out []result
    for {
        part, err := mr.NextPart() // splits the body on the boundary string
        if err == io.EOF {
            break
        }
        if err != nil {
            writeErr(w, http.StatusBadRequest, err.Error())
            return
        }
        n, _ := io.Copy(io.Discard, part) // stream: replace Discard with a file
        out = append(out, result{part.FormName(), part.FileName(), n})
    }
    writeJSON(w, http.StatusCreated, out)
}

```

AVOID / CAREFUL WHEN

Enforce limits: `http.MaxBytesReader` (hard ceiling), a file-type allow-list by sniffing content (not by trusting the filename), and a random server-side file name.

For very large files (video), do not proxy the bytes through your Go server at all: return a **pre-signed URL** so the client uploads straight to object storage (S3/GCS/R2). Resumable uploads and ranges are Part 9.

Large responses: chunked transfer, SSE, and friends

Technique	How it works	Direction	Use it for	Not for
Plain response	Content-Length known, one body	Server → client	Normal APIs	Huge or endless data
Chunked transfer (HTTP/1.1)	No Content-Length; body sent in chunks, ends with a zero chunk. Go does it automatically when you <code>Flush()</code>	Server → client	Generated downloads, streaming JSON lines, exports	Two-way chat
Range requests (206)	Client asks for bytes <code>Range: bytes=0-999</code> ; <code>http.ServeContent</code> supports it	Server → client	Video seek, resumable downloads	Dynamic data
Server-Sent Events	<code>Content-Type: text/event-stream</code> ; long-lived response with <code>data: ...</code> frames; browser <code>EventSource</code> auto-reconnects	Server → client only	Live notifications, progress bars, price ticks, logs	Binary data; client → server messages
WebSocket	Starts as HTTP (101 <code>Switching Protocols</code>), then a two-way message channel	Both	Chat, games, collaborative editing, live auctions	Simple one-way feeds (SSE is simpler)
Long polling	Client asks, server holds the request until data exists	Server → client	Legacy fallback	New designs

code/ch05_http/server/handlers.go

```
// stream pushes Server-Sent Events: one long-lived response, many chunks.
func stream(w http.ResponseWriter, r *http.Request) {
    fl, ok := w.(http.Flusher)
    if !ok {
        writeErr(w, http.StatusInternalServerError, "streaming unsupported")
        return
    }
    h := w.Header()
    h.Set("Content-Type", "text/event-stream")
    h.Set("Cache-Control", "no-cache")
    h.Set("Connection", "keep-alive")

    tick := time.NewTicker(200 * time.Millisecond)
    defer tick.Stop()
    for i := 1; i <= 5; i++ {
        select {
        case <-r.Context().Done(): // client went away: stop working
            return
        case <-tick.C:
            fmt.Fprintf(w, "id: %d\ndata: chunk %d\n\n", i, i) // SSE frame ends with blank line
            fl.Flush()                                         // push it to the client NOW
        }
    }
}
```

Real output: chunks arrive 200 ms apart over one open response

```
$ curl -sN localhost:8082/stream
id: 1
data: chunk 1

id: 2
data: chunk 2
...
```

CORRECTION / NUANCE

The video lumps "chunked transfer" and "text/event-stream" together. They are different layers: **chunked encoding** is how HTTP/1.1 frames a body of unknown length (HTTP/2 has no chunked encoding; it uses DATA frames). **SSE** is a message format and browser API on top of any long-lived response.

`http.Server.WriteTimeout` will cut off long streams. Leave it unset for streaming endpoints, or extend the deadline per request with `http.ResponseController` (Go 1.20+). Your repo's `LimitInFlight` already exempts `/v1/ws` for the same reason.

Always select on `r.Context().Done()` so you stop working when the client disconnects. Nginx buffers by default: add `X-Accel-Buffering: no` for SSE.

12. HTTPS, TLS and certificates

HTTPS = HTTP over TLS. TLS provides three guarantees: **confidentiality** (nobody on the path can read it), **integrity** (nobody can modify it unnoticed) and **authentication** (you are really talking to `api.example.com`, proven by a certificate signed by a trusted authority). SSL is the obsolete predecessor; people still say "SSL certificate" out of habit.

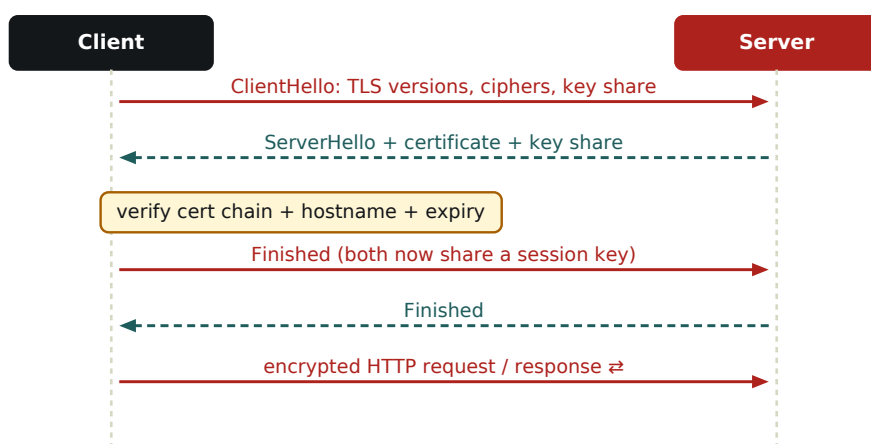


Figure 12.1 Simplified TLS 1.3 handshake. Asymmetric crypto agrees a key; fast symmetric crypto then protects the data.

HTTPS / TLS

WHAT IT IS Encryption, integrity and server identity for HTTP, using certificates issued by a certificate authority (e.g. Let's Encrypt, free and automatic).	WHY WE NEED IT Plain HTTP exposes passwords, tokens and cookies to anyone on the network, and lets them inject content. Browsers also hide powerful features (service workers, geolocation, HTTP/2 and 3) from plain HTTP.
USE IT WHEN Always in production, including internal service-to-service traffic when it crosses networks you do not fully control.	DON'T USE IT / CAREFUL WHEN In local development plain HTTP is fine. Do not hand-roll crypto or disable certificate verification (<code>InsecureSkipVerify: true</code>) "just to make it work".
ADVANTAGES Security, trust, SEO and browser features, HTTP/2 and HTTP/3 support.	DISADVANTAGES Handshake latency, certificate renewal operations, harder to debug traffic (use the proxy's logs or <code>SSLKEYLOGFILE</code>).
HOW (in Go / in practice) Most teams terminate TLS at a reverse proxy / load balancer (Nginx, Caddy, cloud LB) and speak plain HTTP to the Go app inside a private network. Go can also do it directly with <code>ListenAndServeTLS</code> or <code>autocert</code> .	

Two ways to do HTTPS

```
// Option A: TLS inside Go
srv := &http.Server{Addr: ":443", Handler: h, TLSConfig: &tls.Config{MinVersion: tls.VersionTLS12}}
log.Fatal(srv.ListenAndServeTLS("fullchain.pem", "privkey.pem"))

// Option B (most common): Go listens on :8080 behind Nginx/Caddy/LB that handles TLS.
// Then trust X-Forwarded-Proto ONLY from that proxy, and send HSTS from the app or the proxy.
```

IN YOUR auctionEngine REPO

`SecurityHeaders(hsts bool)` in your repo sends `Strict-Transport-Security` only when `HSTS` is enabled for production. Sending it over plain HTTP does nothing, and enabling it too early on a domain you cannot yet serve over HTTPS locks users out for a year.

13. Everything above, inside your auctionEngine repo

Reading production code is the fastest way to turn theory into skill. Here is where each idea from this part lives in your project, so you can open the file and see a real, tested version.



Figure 13.1a Your global middleware chain, outermost first (internal/server/server.go).



Figure 13.1b ...continued. Then the router, then per-route auth, per-user rate limits and the handler.

Concept in this PDF	Where in auctionEngine	What to notice
Handler wrapping, middleware chain	<code>internal/middleware/middlew are.go Chain</code>	First listed is outermost; ordering is a design decision (metrics outermost so it counts the 500s Recover writes)
Method + path routing	<code>internal/server/server.go routes, Routes</code>	<code>mux.Handle(r.Method+" "+r.Pattern, ...)</code> uses Go 1.22 patterns, so wrong methods get 405 for free
Consistent error bodies and JSON helpers	<code>internal/httpx/httpx.go</code>	<code>maxBodyBytes = 1 << 20</code> caps request bodies; one error shape everywhere
Security headers, CORS and pre-flight	<code>internal/middleware/middlew are.go</code>	Allow-list, <code>Vary: Origin</code> , <code>Max-Age</code> , 403 for unknown pre-flights, a CSRF explanation in the comment
Server timeouts	<code>internal/server/server.go New</code>	<code>ReadHeader 5 s</code> , <code>Read 10 s</code> , <code>Write 15 s</code> , <code>Idle 60 s</code>
Idempotency key on a real POST	<code>internal/handlers/bid.go, internal/bidding/bid.go</code>	Bid placement accepts <code>Idempotency-Key</code> ; a reused key with a different body is <code>ErrIdempotencyMismatch</code>
429 and Retry-After	<code>internal/ratelimit/middlewa re.go</code>	Per-IP for auth, per-user for bids
503 and Retry-After (load shedding)	<code>internal/middleware/shed.go</code>	Explains congestion collapse and why rejecting early protects everyone
Liveness vs readiness, draining	<code>internal/health/health.go, cmd/api/main.go</code>	<code>/readyz</code> answers 503 while draining so the load balancer stops sending traffic
Graceful shutdown	<code>cmd/api/main.go (signal.Notify, shutdown)</code>	Stop accepting → drain → wait for workers, all under one 10 s deadline

Concept in this PDF	Where in auctionEngine	What to notice
Long-lived connections	<code>internal/realtime/hub.go</code>	WebSocket connections are hijacked from net/http, so <code>RegisterOnShutdown(hub.Close)</code> is needed
Documented decisions	<code>docs/decisions/0001...0015</code>	ADRs: the why behind outbox, Argon2id, SKIP LOCKED, Kafka bids, gRPC, resilience

INTERVIEW ANGLE

In interviews, **pointing at your own project** ("in my auction engine the bid endpoint takes an Idempotency-Key and rejects a reused key with a different body; the load shedder returns 503 with Retry-After instead of queueing") is far stronger than reciting definitions.

14. Code repository design for this course

The sample code is a Go module that grows chapter by chapter. Every folder is runnable on its own and tested. This mirrors how you will structure real projects: one `go.mod`, one folder per executable under `cmd/` (here, per chapter), shared logic in packages.

Layout

```

backend-course/
├── code/
│   ├── go.mod
│   └── ch05_http/
│       ├── rawtcp/main.go
│       └── server/
│           ├── main.go
│           ├── handlers.go
│           ├── middleware.go
│           ├── idempotency.go
│           └── server_test.go
├── client/
│   ├── client.go
│   └── client_test.go
└── pdf/

```

one Go module: backendcourse (go 1.22+, standard library only)

HTTP by hand over TCP: no magic

http.Server, timeouts, graceful shutdown

CRUD, ETag/304, negotiation, multipart, SSE, redirects

CORS, logging, gzip

Idempotency-Key replay

10 tests: status, idempotency, CORS, cache, gzip, SSE...

timeouts, pooling, body handling

proves keep-alive reuse and context cancel

sources that generate these PDFs (reportlab)

How the pieces fit (the same shape as your auctionEngine):

Layer	Responsibility	In the sample	In auctionEngine
main	Wire config, build the server, handle signals	<code>server/main.go</code>	<code>cmd/api/main.go</code>
Middleware	Cross-cutting: log, CORS, auth, limits, panic recovery	<code>middleware.go</code>	<code>internal/middleware</code>
Handler	Parse + validate input, call service, write response	<code>handlers.go</code>	<code>internal/handlers</code>
Service	Business rules (Part 2)	(added in Part 2)	<code>internal/auction</code> , <code>internal/bidding</code>
Repository	Talks to the database (Part 5)	(added in Part 5)	<code>internal/database</code>

Run it yourself

Commands

```
cd backend-course/code
go test -race ./...           # 13 tests, race detector on
go run ./ch05_http/server     # http://localhost:8082
go run ./ch05_http/rawtcp     # http://localhost:8081 (hand-written HTTP)

# poke it
curl -i localhost:8082/hello -H 'Accept-Language: es'
curl -i -X POST localhost:8082/notes -H 'Idempotency-Key: k1' -d '{"text": "hi"}'
curl -i localhost:8082/notes/1
curl -i -H 'If-None-Match: "<etag from above>"' localhost:8082/notes/1      # 304
curl -i -X OPTIONS localhost:8082/notes/1 -H 'Origin: http://localhost:5173' \
-H 'Access-Control-Request-Method: PUT'                                     # pre-flight
curl -sN localhost:8082/stream                                           # SSE
curl -s -H 'Accept-Encoding: gzip' localhost:8082/big | wc -c           # compressed size
```

15. Beyond the video: what else interviews ask about HTTP

The video is an excellent base. These are the HTTP topics it does not cover (or only names) that come up in interviews and real incidents.

Topic	What you must be able to say
Cookies	Set-Cookie / Cookie ; attributes HttpOnly , Secure , SameSite , Domain , Path , Max-Age ; cookies are sent automatically by the browser, which is exactly what makes CSRF possible. Header-based tokens are not sent automatically.
Redirect loops and 307/308	Method preservation; Location header; caching of 301; open-redirect vulnerability when you redirect to a user-supplied URL.
Reverse proxy / forward proxy / load balancer	Reverse proxy sits in front of servers (TLS, caching, routing); forward proxy sits in front of clients. L4 vs L7 balancing. Real client IP comes from X-Forwarded-For ; trust it only from known proxies.
DNS	Domain → IP lookup; TTL and caching; A/AAAA/CNAME; why a deploy "works for some users" (stale DNS).
Timeouts and Slowloris	ReadHeaderTimeout defeats clients that dribble headers byte by byte to hold connections open.
Head-of-line blocking	HTTP/1.1 at the HTTP level, HTTP/2 at the TCP level, solved in HTTP/3 (QUIC).
Trailers and 100-continue	Expect: 100-continue lets a client ask permission before sending a large body; rarely needed but appears in interviews.
Retry-After and back-off	429/503 should include Retry-After ; clients retry with exponential back-off and jitter , only for idempotent operations.
Request size and abuse limits	Cap body, header size and URL length; reject early. MaxBytesReader , MaxHeaderBytes .
Content-Disposition and downloads	attachment ; filename=... for downloads; sanitise filenames; set a correct Content-Type .
HTTP/2 and gRPC	gRPC needs HTTP/2 because it relies on multiplexed streams and trailers (Part 9).
Observability headers	X-Request-ID / traceparent propagate a trace id across services; return it in responses so support can find the exact request.

PART 1 · FACT QUESTIONS, THEN SITUATIONS

16. Interview preparation

A. Core questions (the video's, refined)

1. What does "HTTP is stateless" mean, and how do you keep a user logged in?

The server keeps no memory between requests; each request must be self-contained. Login continuity is simulated by sending a session cookie or a bearer token with every request, and the server looks up or verifies it each time. That statelessness is what lets any replica serve any request.

2. PUT vs PATCH?

PUT replaces the whole resource with the body (idempotent). PATCH applies a partial change (not guaranteed idempotent). Use PATCH for typical updates; PUT when the client owns the id or wants a full replace.

3. What is idempotency and which methods are idempotent?

Repeating the request leaves the server in the same state as sending it once. GET, HEAD, PUT, DELETE and OPTIONS are; POST is not; PATCH depends on the operation. It is about state, not responses: a second DELETE may return 404.

4. Your React app on :3000 calls your API on :3001 and the browser blocks it. Why, and how do you fix it?

Different ports mean different origins, so the same-origin policy hides the response unless the API opts in. Fix it on the **server**: return `Access-Control-Allow-Origin: http://localhost:3000` (an allow-list, not `*`), and answer the OPTIONS pre-flight with 204 plus `Allow-Methods`, `Allow-Headers` and `Max-Age`. Alternatively proxy `/api` through the dev server so it is same-origin.

5. When does the browser send a pre-flight request?

Only for cross-origin requests that are not "simple": method other than GET/HEAD/POST, or a non-safelisted header (`Authorization`, custom headers), or a `Content-Type` other than form-urlencoded, multipart or text/plain (so JSON always pre-flights).

6. Explain HTTP caching with ETag and 304.

The server returns a validator (`ETag`) and freshness info (`Cache-Control`). When the copy is stale the client sends `If-None-Match`. If it still matches, the server replies 304 with no body and the client reuses its copy; otherwise 200 with a new ETag.

7. You get 502 Bad Gateway. What does it mean?

A proxy or load balancer received an invalid response (or none: connection refused) from the upstream app. Check that the app is running, listening on the expected port, and not crashing. 504 means the upstream was too slow.

8. What is content negotiation?

The client states preferred format, language and encoding with `Accept`, `Accept-Language`, `Accept-Encoding`; the server picks the best match, reports it with `Content-Type`, `Content-Language`, `Content-Encoding`, and sets `Vary`. If nothing matches: 406.

9. How would you let users upload a 2 GB video?

Do not send it as JSON and do not buffer it in memory. Use multipart streaming with a hard size limit, or better, issue a **pre-signed URL** so the client uploads directly to object storage, with resumable or chunked upload for reliability; process it asynchronously with a queue.

10. How do you push live logs from the server to the browser?

Server-Sent Events: `Content-Type: text/event-stream`, flush each event, stop on `r.Context().Done()`. If the client must also send messages, use WebSocket. Mind proxy buffering and server write timeouts.

B. Deeper protocol questions

11. Why does HTTP/2 not fully fix head-of-line blocking, and what does HTTP/3 change?

HTTP/2 multiplexes streams over **one TCP connection**; TCP delivers bytes in order, so one lost packet stalls every stream. HTTP/3 uses QUIC over UDP with independent streams, so only the affected stream waits; it also merges transport and TLS handshakes.

12. 401 or 403 for "valid token but not an admin"? And for "no token"?

No/invalid token: 401 (unauthenticated, with `WWW-Authenticate`). Valid token, insufficient role: 403. For resources the user must not even know about, 404.

13. Difference between `no-cache` and `no-store`?

`no-cache` allows storing but forces revalidation before reuse. `no-store` forbids storing at all. Use `no-store` for sensitive data.

14. What is the difference between a reverse proxy and a forward proxy?

A forward proxy acts for clients (corporate proxy, VPN). A reverse proxy acts for servers (Nginx, load balancer, CDN): terminates TLS, balances load, caches, hides backends.

15. Is CORS a security feature that protects my API?

No. It is a browser rule that protects **users** from other sites reading their data. Attackers' own scripts, curl and bots ignore it. Protect the API with authentication, authorisation, and rate limits.

16. What is `Vary` and why does it matter?

It lists request headers that influenced the response (`Origin`, `Accept-Encoding`, `Accept-Language`). Caches must include them in the cache key; forgetting it lets a CDN serve a gzip body to a client that cannot decode it, or one origin's CORS header to another.

17. Why must CORS middleware run before authentication?

Pre-flight OPTIONS requests carry no credentials. If auth runs first it returns 401 and the browser reports a CORS failure, so the real request is never sent.

18. What goes wrong with `http.Get` in production?

No timeout (hung goroutines, cascading failure), possible connection leaks if the body is not closed, and pool starvation with the default 2 idle connections per host. Use a shared client with timeouts, contexts and tuned transport.

19. What does `ReadHeaderTimeout` protect against?

Slowloris: a client opens many connections and trickles header bytes to exhaust the server's connections. A short header timeout closes them.

20. TLS handshake: what is it for, and what does the client verify?

To agree on keys and authenticate the server. The client checks the certificate chain leads to a trusted CA, the hostname matches, and the certificate is within its validity period.

C. Situation and scenario questions

21. The user clicks "Place bid" twice, or the mobile app retries after a timeout. How do you avoid two bids?

Make the operation idempotent. The client generates an `Idempotency-Key` once per user intent and reuses it on retries; the server stores key → result (Redis/DB, with TTL, bound to the user) and replays the stored response. Same key with a different body is rejected (422/409). Handle two identical requests arriving simultaneously with a lock or unique constraint. Your auction engine does this in the bid path.

22. After a frontend deploy some users see the old UI, others the new one. Why?

Caching: the browser or CDN holds an old `index.html` or old JS. Serve HTML with `no-cache` (revalidate), serve JS/CSS with content-hashed names and `max-age=31536000`, `immutable`, and purge the CDN entry point if needed.

23. Every API call from the browser fails with a CORS error, but curl works. Walk me through the debugging.

1) Open the Network tab: is there a failed OPTIONS? 2) Compare `Origin` with the server allow-list (scheme, host, port, trailing slash). 3) Check the pre-flight reply: status 2xx, `Allow-Methods/Allow-Headers` contain what was requested. 4) Check error responses also carry CORS headers. 5) Check auth middleware is not rejecting OPTIONS. 6) With cookies: no `*`, `Allow-Credentials: true`, and `SameSite` settings.

24. Latency doubled but CPU is idle. Where do you look?

Connection handling and dependencies, not compute: missing keep-alive or tiny client pool (new TCP/TLS per call), DNS lookups, a slow downstream with no timeout, DB pool exhaustion, a proxy buffering, or a TLS handshake storm after a deploy. Check traces and per-phase timings (DNS, connect, TLS, first byte).

25. Your service returns 502 for about 30 seconds on every deploy. Why and how do you fix it?

Instances are killed while still receiving traffic or the new one is not ready. Fix: a readiness endpoint that fails while draining, a delay before closing the listener, graceful **Shutdown** that finishes in-flight requests, and rolling deploys that wait for readiness. This is exactly what `cmd/api/main.go` implements.

26. A partner needs to download a 5 GB report. How do you serve it?

Generate it asynchronously (202 Accepted + job id), store it in object storage, and return a time-limited pre-signed URL. If you must stream from the app: chunked response with flushing, **Content-Disposition**, support **Range** for resume, no server **WriteTimeout** on that route, and cancel work on client disconnect.

27. You must show live auction prices to 50,000 viewers. HTTP polling, SSE or WebSocket?

Viewers only receive, so SSE or WebSocket both fit; polling every second means 50,000 requests per second of pure overhead. Choose WebSocket if you need client messages or already run it; SSE for simplicity. Either needs a fan-out layer (pub/sub through Redis/Kafka) so any instance can push, connection limits and heartbeats, and shedding. See Part 7.

28. An endpoint sometimes returns 200 with an error message in the body. What is wrong and why does it matter?

It breaks the contract: caches may store it, monitors count it as success, clients' retry logic and generated SDKs mis-handle it. Use the right 4xx/5xx and a consistent error body.

29. How would you rate limit and what do you return?

Identify the caller (user id after auth, otherwise IP via a trusted proxy header), apply a token-bucket or sliding window (shared in Redis when multiple replicas), return 429 with **Retry-After** and rate-limit headers. Separate limits for login, bids and reads. (Part 10.)

30. Design the HTTP behaviour of a payment endpoint.

POST with **Idempotency-Key**, TLS only, strict input validation, 201 with **Location** or 202 if async, **Cache-Control: no-store**, 409/422 on key reuse mismatch, never log card data, timeouts at every hop, webhooks to confirm final state with signature verification, and a reconciliation job. Retries by the client are safe only because of the key.

17. One-page cheat sheet

I need to...	Do this
Create a resource	<code>POST /things</code> → 201 + <code>Location: /things/42</code>
Replace / partially update	<code>PUT /things/42</code> (full) · <code>PATCH /things/42</code> (partial) → 200 or 204
Delete	<code>DELETE /things/42</code> → 204 (repeat → 404 is fine)
Make POST retry-safe	<code>Idempotency-Key</code> header + stored response
Bad input	400 (or 422) + <code>{"error": "...", "fields": {...}}</code>
Not logged in / not allowed	401 (+ <code>WWW-Authenticate</code>) / 403 (or 404 to hide)
Duplicate or edit collision	409 (or 412 with <code>If-Match</code>)
Too many requests	429 + <code>Retry-After</code>
Overloaded or draining	503 + <code>Retry-After</code>
Cache a public read	<code>Cache-Control: public, max-age=N</code> + <code>ETag</code> ; honour <code>If-None-Match</code> → 304
Never cache	<code>Cache-Control: no-store</code>
Support a browser on another origin	CORS allow-list middleware, <code>Vary: Origin</code> , answer OPTIONS with 204, run before auth
Serve many formats / languages	Read <code>Accept*</code> , set <code>Content-*</code> and <code>Vary</code> , 406 if none
Big download / upload	Stream; pre-signed URLs; <code>MaxBytesReader</code> ; <code>Range</code> support
Live updates server → client	SSE (<code>text/event-stream</code> , flush, <code>r.Context().Done()</code>) or WebSocket
Outgoing HTTP call	Shared <code>http.Client</code> with timeouts, context, <code>defer resp.Body.Close()</code> , check status
Server hardening	<code>ReadHeaderTimeout</code> , <code>ReadTimeout</code> , <code>IdleTimeout</code> , body size cap, graceful <code>Shutdown</code> , security headers

Exercises (do these before Part 2)

- Run the raw TCP server and send a request with `nc`. Add a `POST /echo` route that returns the request body (you must read `Content-Length` bytes).
- Add `If-Match` support to `PUT /notes/{id}`: return **412** when the ETag does not match (optimistic locking, the fix for lost updates). Write the test first.
- Add `Retry-After` and a **429** to the sample server with a small token bucket per client IP.
- Make `GET /notes` return a list with `Link`-based or cursor pagination and a weak ETag for the list.
- Serve a file with `http.ServeContent` and verify `Range: bytes=0-99` returns **206** with `Content-Range`.
- In the browser, from a page on `http://localhost:5173`, call `PUT http://localhost:8082/notes/1` with JSON and watch the OPTIONS pre-flight in the Network tab; then remove one CORS header and read the error.
- Read `internal/middleware/shed.go` in `auctionEngine` and write, in your own words, why returning 503 immediately is better than queuing.

REMEMBER

Coming next, Part 2: routing, serialization/deserialization (JSON, protobuf, encoding pitfalls), validation and transformation, and the controller → service → repository layering with `context`, using Go code you can run. Paste the transcripts of the next videos and I will check the PDF against them.